

Aula 2 - Vetores (Parte 1)

Em alguns determinados problemas, precisamos armazenar muitas informações simultaneamente. O procedimento padrão para armazenar informações é o uso de variáveis, porém, a partir de um certo ponto, elas tornam-se inviáveis.

Para resolver esse tipo de problema, usamos **vetores**. Em C++, temos duas estruturas de dados referentes a vetores: os **arrays** e os **vectors**. Essa aula engloba apenas arrays.

Operações básicas com arrays

```
int meu_array[10];
```

Essa linha reserva 10 espaços de memória, indexados (numerados) de 0 a 9, inicialmente vazios (ou seja, contém lixo de memória), e dedicados para armazenar números inteiros.

É como se tivéssemos 10 variáveis diferentes:

```
meu_array[0]
meu_array[1]
meu_array[2]
...
meu_array[8]
meu_array[9]
```

Podemos acessar cada um deles como se fossem variáveis normais, alguns exemplos:

```
cout << meu_array[6]; // mostra na tela o sétimo elemento d
o vetor chamado meu_array
meu_array[3] = 9; // na posição 3 do vetor agora está armazenado o valor 9
meu_array[1]++; // aumenta 1 no número armazenado na segunda posição do vetor
```

```
if(meu_array[1] > meu_array[2]) // usa os elementos armazenados para uma comparação
```

Os vetores, por causa de sua natureza sequencial e repetitiva, normalmente acompanham-se de loops. Em problemas que necessitam que você peça ao usuário um vetor de números, usamos loops para pedi-los:

```
for(int i = 0; i < 10; ++i){  
    cin >> meu_array[i];  
}
```

Note que, como `i` varia de 0 até 9, nós passamos por todas as posições do vetor, uma de cada vez.

Vale ressaltar que podemos declarar vetores contendo qualquer tipo de variável:

```
bool verdadeiroOuFalso[30];  
float reais[1000];
```

Então, via de regra:

```
<tipo_do_vetor> <nome_do_vetor>[<tamanho>];
```

Na maioria dos problemas, trabalhamos com um vetor de “tamanho N”, isso quer dizer: o tamanho do vetor, embora fixo na hora da execução, vai variar de acordo com a entrada. Para isso, precisamos, primeiro, pedir o tamanho do vetor no começo do problema, antes de começar a pedir o próprio vetor:

```
int tamanho;  
cin >> tamanho;  
  
int vetor[tamanho];  
for(int i = 0; i < tamanho; ++i){  
    cin >> vetor[i];  
}
```

Observações importantes

1. Sempre preenchem os vetores antes de usá-los (mesma coisa que com variáveis comuns), para evitar de acessarmos lixo de memória.
2. Cuidado com índices: acessar um índice fora do vetor (índice negativo ou maior que o tamanho do vetor) é fatal pra maioria dos problemas.
3. Nunca esquecer que vetores são indexados a partir do 0.
4. Não é possível dar um `cout << vetor;` tal qual no python, precisamos mostrar elemento por elemento.

Strings

Na primeira aula, citamos como tipo de variável as **strings**. Agora, com a nova percepção de vetores, podemos dizer que as strings nada mais são do que um vetor de **chars** (caracteres).

Isso quer dizer que podemos tratá-las da mesma maneira que faríamos com vetores:

```
string nome = "Gabriel Matias";
cout << nome[0]; // vai mostrar 'G'

for(int i = 0; i<14; ++i){
    cout << nome[i];
} // vai mostrar o nome inteiro, letra por letra

cout << nome; // embora tenha a natureza de um vetor, e eu
tenha dito antes que
// cout << vetor não funciona, cout << string funciona
```

As strings também têm um operador importante, podemos concatenar (isto é, juntar) strings com o operador de soma:

```
string nome_completo = nome + " de Almeida";
cout << nome_completo; // vai mostrar "Gabriel Matias de Almeida"
```